University of Pisa

Parallel and Distributed Systems

Optimization of Partitioned Hash Join with Duplicates

Assignment 2 - Report

Gianluca Panzani

April 18, 2026

Contents

1 Implementation **1**
1.1 Hash Function Choice 1
1.2 Benchmark 1

2 Parallelization Strategy **1**
2.1 Partition and join phases 1
2.2 What remains mostly sequential 2

3 Results Analysis **2**
3.1 Join and Partition Time 2
3.2 Speedup 3
3.3 Strong Scaling 4
3.4 Weak Scaling 5

4 Correctness validation **6**

5 Conclusions **6**

1 Implementation

The repository is organized around one sequential baseline and four optimized variants of the same partitioned hash-join pipeline. The core source files are `hashjoin_seq.cpp` (sequential reference, baseline), `hashjoin_par_p.cpp` (parallel partition phase, sequential join phase), `hashjoin_par_pj.cpp` (parallel partition + parallel join, with contiguous partition ranges), `hashjoin_par_pj_wb.cpp` (adds workload balancing of partitions in the join phase), and `hashjoin_par_pj_wb_map.cpp` (same balanced scheduling, plus flatter memory layout for partition counters/offsets and a custom flat hash table in the join build phase).

All executables preserve the same algorithmic structure: deterministic generation of relations `R` and `S`, partitioning of both relations (histogram, exclusive prefix sum, scatter), local build/probe join on matching partition ids, and reduction into `join_count`, `checksum1`, `checksum2`. Keeping this structure unchanged across versions is important because it allows direct and fair comparisons between optimization steps. Phase timings are exported separately (partition, join and total), so each code change can be analyzed also at phase level.

1.1 Hash Function Choice

In Module 1, the simple `mask` mapping was the fastest option in my tests. However, because of requirments, i haven't kept the same mapping in this module. So I switched to a `xorshift`-based mapping before masking, which is not as fast as the previous one, but is still efficient and effective.

I did not choose the `fmix32fold`-style mapping here because it is heavier in arithmetic operations and, in previous experiments, it gave worse performance than `xorshift`.

1.2 Benchmark

Benchmark automation is implemented in `benchmark.sh`. The script takes as input the executable target, the grid serch file and executes all combinations via Slurm (`run_with_slurm.sh`). For each run, the executable appends one CSV row containing configuration parameters, phase times, speedup and so on.

In the current campaign, fixed values are `nr=50'000'000`, `ns=50'000'000`, `P=256`, `seed=13`, `max_key=1'000'000`; `partition_threads` and `join_threads` vary over $\{4, 8, 16, 32, 64\}$ for the parallelized versions, while the sequential baseline runs with one thread.

The Slurm runner (`runners/run_with_slurm.sh`) is built to run a single combination of parameters of the grid and pins execution to `node05`, avoiding machine-level variability.

2 Parallelization Strategy

Parallelization is introduced phase-by-phase, without changing the global algorithm.

2.1 Partition and join phases

For each relation, the partition pipeline, which is parallelized in all the parallel executable variants, keeps the standard three stages:

1. *Histogram:* input is split into chunks and each thread builds a local histogram over P partitions.

2. *Prefix sum:* local histograms are reduced into a global histogram, then an exclusive prefix sum computes global partition starts. In the current code this step is **not parallelized**: it is executed as a serial scan over the P counters, followed by a serial construction of per-thread offsets.

3. *Scatter:* each thread writes its chunk into the output using thread-specific offsets derived from the prefix stage.

The key design detail is that scatter uses precomputed thread-local offsets, so the hot loop does not require atomics or locks.

After partitioning, partitions are independent units of work and, in this way, also the join phase doesn't need locks. So, at the end, the code is organized like follows:

- `hashjoin_seq`: join and partition phases are both sequential.
- `hashjoin_par_p`: join is still sequential; only partition is parallel.
- `hashjoin_par_pj`: partitions are split into contiguous ranges across join threads.
- `hashjoin_par_pj_wb`: partitions are assigned with greedy load balancing based on their size.
- `hashjoin_par_pj_wb_map`: same balanced scheduling, plus a flat open-addressing count table in build/probe and flat arrays in partition counters/offsets.

2.2 What remains mostly sequential

The global prefix-sum step and the final reduction of per-thread join results are lightweight compared with histogram/scatter/probe loops, so they are kept simple. In practice, with $P = 256$, this serial part has low absolute cost compared with the full partition and join loops. This is coherent with the measured behavior: once partition and join loops are parallelized, remaining bottlenecks come mostly from memory traffic and per-thread overhead at high thread counts, not from control-heavy serial code.

3 Results Analysis

All plots in this section are generated from the benchmark CSV files in `src/results/`. The sequential baseline mean total time is 4.191 s.

Note that in the matrix-like plots of this section we have that:

- the bars in each position of the matrix represent the value of `seq`, `par_p`, `par_pj`, `par_pj_wb`, `par_pj_wb_map` (from left to right),
- the number of `join_threads` and `part_threads`, which are on the two axes, vary over $\{1, 4, 8, 16, 32, 64\}$.

3.1 Join and Partition Time

The phase-level matrices at the end of the section, show the central behavior of the implementation.

Partition time scales well from 4 to 32 threads, then flattens around 0.92 s at 4 partition threads and around 0.36–0.37 s at 32–64 partition threads. This indicates good early scaling followed by memory-system saturation.

Join time is where variants differ the most. The executables `par_pj` and `par_pj_wb` reduce join time to about 0.18 s in the best area. The `par_pj_wb_map` variant improves further, reaching 0.13 s (best run, 32 partition threads and 16 join threads), confirming that data-structure locality in build/probe is a major factor.

As we can see, partition time is similar across all parallel variants (`par_p`, `par_pj`, `par_pj_wb`, `par_pj_wb_map`) because they use the same partition implementation, in fact the main variation comes from thread count. In contrast, once the join phase is parallelized (`par_pj`, `par_pj_wb`,

par_pj_wb_map), join time differences become more evident, with a clear gap between par_pj_wb and par_pj_wb_map.

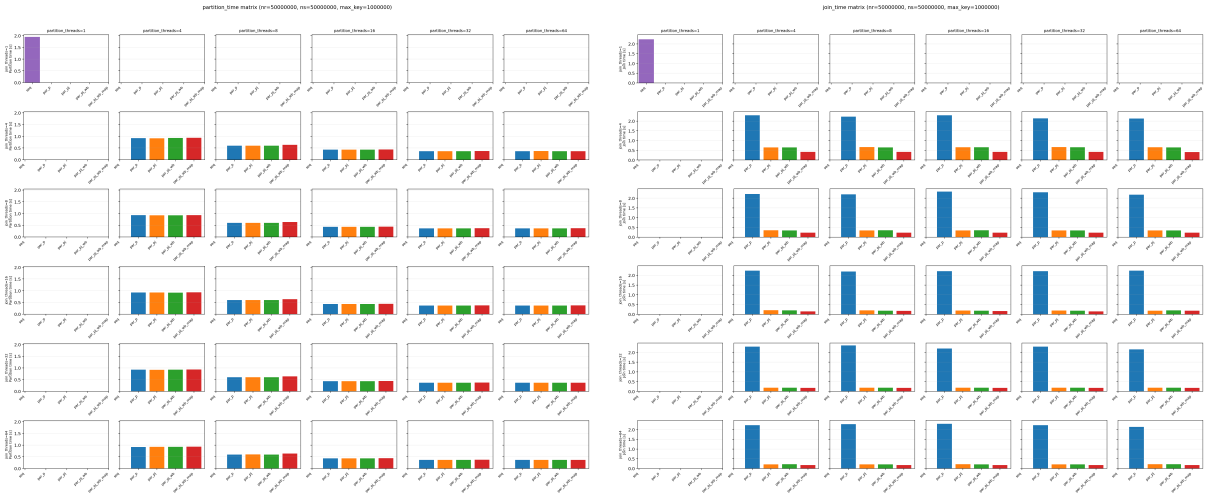

Figure 1: Partition-time and join-time matrices for each combination of the number of join threads (rows of the matrix) and partition threads (columns of the matrix).

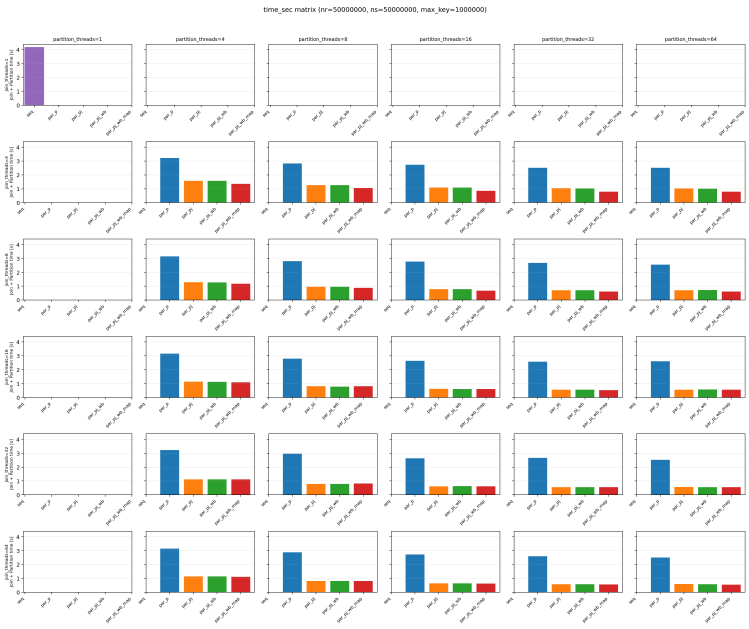

Figure 2: Total execution-time (time_sec) matrix for each combination of the number of join threads (rows of the matrix) and partition threads (columns of the matrix).

3.2 Speedup

Global speedup is computed against the sequential baseline.

The table confirms the optimization trajectory: partition-only parallelization gives limited gains, while parallel join plus balanced scheduling gives the main jump; flatter data structures in par_pj_wb_map provide the final improvement.

Variant	Best T_{part}	Best T_{join}	Best time [s]	Speedup
`hashjoin_par_p`	64	32	2.394	1.75×
`hashjoin_par_pj`	64	16	0.549	7.63×
`hashjoin_par_pj_wb`	32	16	0.545	7.69×
`hashjoin_par_pj_wb_map`	32	16	0.507	8.26×

Table 1: Best observed configuration per executable.

The gain of workload balancing alone (`par_pj_wb` vs `par_pj`) is small in this campaign. With $P = 256$, $NS = NR = 5 \cdot 10^7$ (which means that the total number of elements is 10^8) and 32 join threads, each thread still processes a lot of partitions. So a first observation could be that the workload could be close to the average even with contiguous assignment. Another one, instead, could be that the size of the partitions is very similar, so the balancing isn't much useful. For this reasons, balancing has limited improvement and the extra speedup is almost absent. A larger benefit could be expected when the dataset is smaller or when effective workload is concentrated in fewer hot partitions.

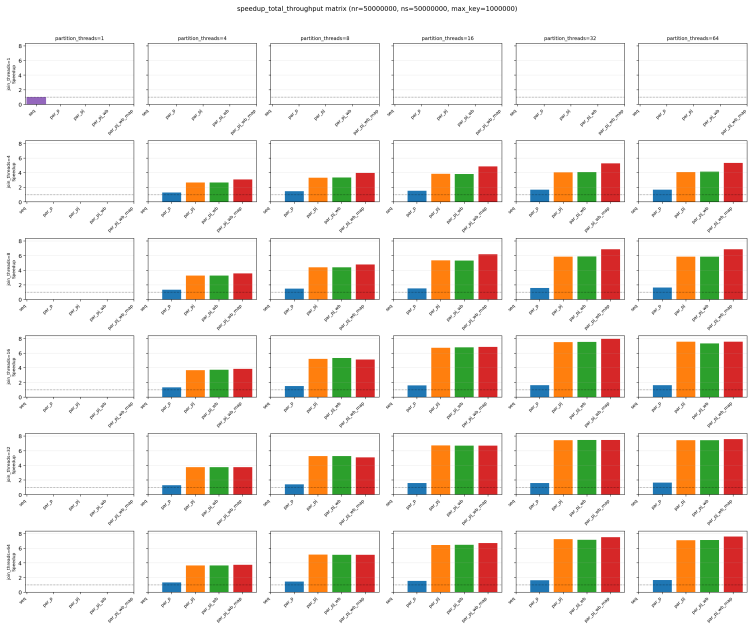

Figure 3: Total speedup matrix over thread combinations and variants.

3.3 Strong Scaling

Strong scaling is measured by fixing input size and increasing threads.

On the diagonal ($T_{part} = T_{join}$, with T number of threads), `par_pj_wb_map` reaches:

- speedup 3.09× at 4 threads;
- speedup 6.87× at 16 threads;
- speedup 7.62× at 64 threads.

For the best implementations, strong scaling improves well up to roughly 16–32 threads, then a clear plateau appears. This suggests that computation is no longer the dominant cost at high thread counts and execution becomes constrained by hardware/system limits.

In fact, the best result is got with `par_pj_wb_map` using 16 join threads and 32 partition threads, with a speedup of about 8.26×.

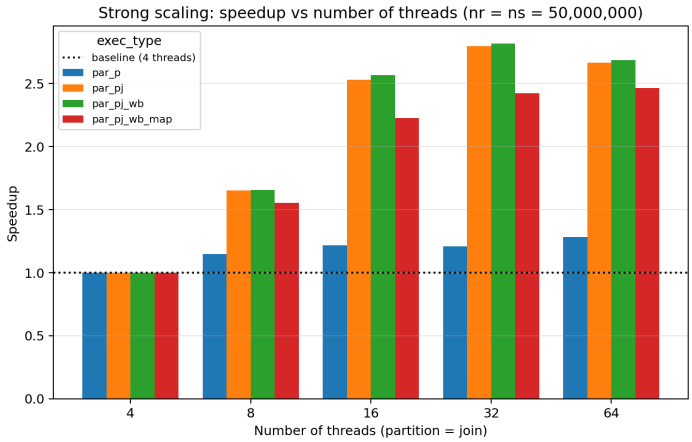

Figure 4: Strong-scaling speedup versus number of threads for each executable type.

3.4 Weak Scaling

Weak-scaling results are now measured with a dedicated campaign (`weak_scaling.cpp`) and grid: $N \in \{10, 20, 40, 80\}$ million records per relation and $T \in \{4, 8, 16, 32\}$ (with $T = T_{part} = T_{join}$). Each (N, T) point is repeated 5 times, then averaged to avoid unstable measures.

The weak-scaling curves are consistent across input sizes. For each executable we have:

- speedup 1.65× at 8 threads;
- speedup 2.36–2.55× at 16 threads;
- speedup 2.64–2.82× at 32 threads.

Overall, weak scalability is only moderate: speedup increases with thread count but clearly sublinearly. Since larger inputs do not preserve near-constant execution efficiency, the main limitation appears to come from machine-level bottlenecks that become more visible at scale, most likely memory bandwidth, cache contention and synchronization overhead.

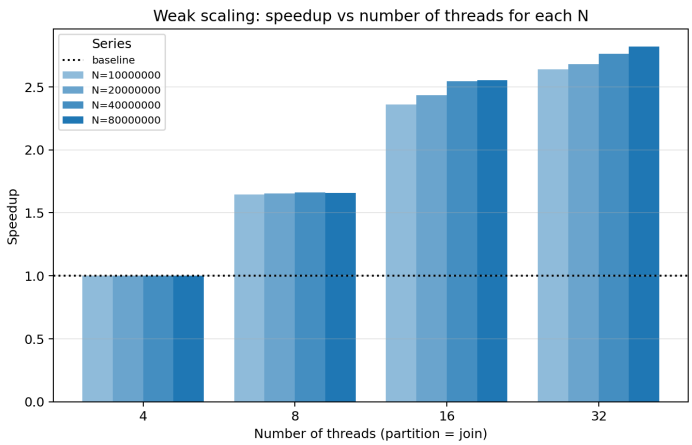

Figure 5: Weak-scaling speedup versus number of threads for each N.

4 Correctness validation

Correctness checks are handled by `checker.cpp`. The checker scans the output directory (`out/` by default), reads all regular files, and compares their raw content byte-by-byte against the first file. The logic is intentionally strict: if at least one file differs, it reports the mismatch and returns a non-OK status.

In this project, each Slurm output file stores the final triple: `join_count`, `checksum1`, `checksum2`. Therefore, identical files imply identical final join outputs across compared runs. In the collected data, all variants produced the same values, which confirms that optimization changes preserve functional correctness.

5 Conclusions

The results show a clear progression:

- parallelizing only partitioning is not sufficient (speedup capped around 1.75×);
- parallelizing join is the dominant improvement (around 7.6×);
- workload balancing improves robustness, but in this setup its standalone speedup is small because average work per thread is already similar;
- locality-oriented structures (`par_pj_wb_map`) deliver the best result (8.26×).

So, for this workload, the main insights are that join-phase optimization dominates total performance, while partition scaling saturates earlier and sets a practical upper bound unless data movement costs are further reduced. Strong scaling is good up to about 16–32 threads for the best variants, then plateaus as shared machine resources saturate. The weak-scaling campaign is only moderate (sublinear speedup growth), which is consistent with machine-level bottlenecks becoming dominant at higher thread counts.